

INTERNAL WORKING OF LIST FUNCTIONS

List Methods:

1.append():

append() is a built-in list method that adds one element at the end of the list.

Syntax:

```
list_name.append(element)
```

Working:

Step 1: List Creation

Step 2: Calling append()

Step 3: Python performs: Check Capacity

If space exists then insert Element and Update the Size of the list

Step 4: capacity is full then:

I. Python allocates a larger memory block.

II. Copy Existing Elements

III. Insert New Element

IV. Delete Old Memory

2.insert():

`insert()` is a built-in list method that inserts an element at a **specific position (index)**.

Syntax:

```
list_name.insert(index, element)
```

Working:

Step 1: List Creation

Step 2: call `insert()`

Step 3: Python performs: Check Capacity

If space exists then Shift Elements Right , Insert New Element and Update Size.

Step 4: capacity is full then:

I. Python allocates a larger memory block.

II. Copy Existing Elements

III. Shift Elements Right

IV. Insert New Element

V. Delete Old Memory

3.pop():

`pop()` removes and returns an element from a list.

Syntax:

```
list_name.pop()
```

Working:

Step 1: List Creation

Step 2: call pop()

Step3: remove element and store removed value

Step4:shift elements to left

Step4:reduce size

4.remove():

`remove( )` removes the **first occurrence** of a specified value from a list.

Syntax:

`list_name.remove(value)`

Working:

Step 1: List Creation

Step 2: call remove()

Step3:Search for the Value

Step4:remove element and Shift Elements Left

Step5:reduce size

5.extend():

`extend( )` is a built-in list method that adds **all elements of an iterable** (list, tuple, set, string, etc.) to the end of a list.

Syntax:

`list_name.extend(iterable)`

Working:

Step 1: List Creation

Step 2: call `extend()`

Step 3: Python performs: Check Capacity

If space exists then insert Element and Update the Size of the list

Step 4: capacity is full then:

I. Python allocates a larger memory block.

II. Copy Existing Elements

III. Insert New Element and list

IV. Delete Old Memory

6.reverse():

`reverse()` is a built-in list method that **reverses the order of elements in the same list.**

Syntax:

`list_name.reverse()`

Working:

Step 1: List Creation

Step 2: call `reverse()`

Step3: using two pointers python Swap first and last elements

Step4: Swap second and fourth elements.

Step5:Pointers meet then Stop and list is now reversed.

7.count():

`count()` is a built-in list method that counts how many times a specified value appears in a list.

Syntax:

```
list_name.count(value)
```

Working:

Step 1: List Creation

Step 2: call count()

Step3:Initialize Counter

Step4:Traverse List

Step5:Return Count

8.sort():

`sort()` is a built-in list method that arranges the elements of a list in **ascending order by default**.

Syntax:

```
list_name.sort()
```

Working:

Step 1: List Creation

Step 2: call sort()

Step3:Find Small Sorted Blocks (Runs)

Step4:Sort Small Runs

Step5:Merge Runs